

SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE:

CSA3712- COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3

5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3

11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3

18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3



❖ **Code of Conduct:**

- ❖ *I AATHISRI.P(192521252) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
- ❖ *I understand that any violation of academic integrity rules will result in disciplinary action.*

❖ **Signature of the student:**

AATHISRI.P
(192521252)

TESTING AS AN ENGINEERING ACTIVITY – BANKING APPLICATION

❖ INTRODUCTION

Software testing should not be treated as a random activity performed only after coding. It should be treated as an **engineering activity** because testing requires planning, systematic methods, tools, standards, measurement, and continuous improvement. In a banking application, unstructured testing can result in serious problems such as incorrect transactions, wrong account balances, duplicate payments, and security issues.

❖ SYSTEMATIC TEST DESIGN

The banking team should first prepare a **test plan** based on requirements. Every important requirement should be converted into test conditions and test cases.

For example, for a fund transfer function, test cases should include:

- ❖ Valid account number and sufficient balance.
- ❖ Invalid account number.
- ❖ Insufficient balance.
- ❖ Transfer amount equal to zero.
- ❖ Maximum allowed transfer amount.
- ❖ Duplicate transaction.
- ❖ Network failure during transfer.
- ❖ Session timeout during transaction.
- ❖ Test cases should contain **test data, preconditions, steps, expected result, and actual result**.

TESTING TOOLS

- ❖ Testing tools can improve efficiency and accuracy. For example:
- ❖ **Jira** or similar tools can be used for defect tracking.
- ❖ **Selenium** can automate web-based banking tests.
- ❖ **Postman** can test banking APIs.
- ❖ **JMeter** can perform performance and load testing.
- ❖ CI tools can automatically execute regression tests after new builds.

STANDARDS AND PROCESSES

- ❖ The organization can follow recognized software testing practices and standards such as **ISO/IEC/IEEE 29119** for software testing processes and documentation. Test reviews, requirement traceability, defect management, and test reports should be standardized.

SUITABLE TECHNIQUES

- ❖ Instead of ad-hoc testing, the team should use:
- ❖ Equivalence partitioning
- ❖ Boundary value analysis

- ❖ Decision table testing
- ❖ State transition testing
- ❖ Risk-based testing
- ❖ Regression testing
- ❖ Exploratory testing
- ❖ Automated testing

PRACTICAL EXAMPLE

- ❖ Suppose the maximum online transfer limit is ₹50,000. Instead of randomly testing amounts, boundary testing can check:
- ❖ **₹49,999, ₹50,000 and ₹50,001.**
- ❖ This is more likely to detect errors in the limit-validation logic.

CONCLUSION

- ❖ Treating testing as an engineering activity makes testing **planned, measurable, repeatable, and systematic**. It improves defect detection and reduces production failures. In banking applications, this approach increases reliability, security, customer confidence, and overall software quality.

2. STRUCTURED TESTING PROCESS FOR AN ATM WITHDRAWAL SYSTEM

INTRODUCTION

- ❖ An ATM is a critical system because it handles money and personal information. A lack of a defined testing process can cause inconsistent results such as incorrect cash dispensing, wrong balance updates, or failed transactions. Therefore, testing should be integrated throughout the **Software Development Life Cycle (SDLC)**.

TESTING ACROSS SDLC PHASES

1. Requirement Analysis

- ❖ The tester studies requirements such as:
 - ❖ User must enter a valid PIN.
 - ❖ Account must have sufficient balance.
 - ❖ ATM should dispense the requested amount.
 - ❖ Account balance must be updated correctly.
 - ❖ Transaction should be recorded.
- ❖ Ambiguous requirements should be identified early.

2. Test Planning

- ❖ The test team decides:
 - ❖ What will be tested.
 - ❖ Testing techniques.
 - ❖ Required test data.
 - ❖ Resources and responsibilities.
 - ❖ Schedule.
 - ❖ Entry and exit criteria.

3. Test Design

- ❖ Test cases are created from requirements.
- ❖ Example:
 - ❖ **Input:** Valid card, valid PIN, balance ₹10,000, withdrawal ₹2,000.
 - ❖ **Expected result:** ATM dispenses ₹2,000 and account balance becomes ₹8,000.

4. Test Execution

- ❖ The test cases are executed and actual results are compared with expected results.

5. Defect Reporting

- ❖ If the ATM deducts ₹2,000 but does not dispense cash, the tester reports a defect with clear steps,

expected result, actual result, severity, and evidence.

6. Retesting and Regression Testing

- ❖ After developers fix the defect, the failed test is executed again. Related ATM functions are also tested to ensure the fix has not introduced another problem.

APPLICATION TO ATM WITHDRAWAL

- ❖ Important test cases include:
- ❖ Valid withdrawal.
- ❖ Invalid PIN.
- ❖ Blocked card.
- ❖ Insufficient balance.
- ❖ Withdrawal exceeding daily limit.
- ❖ Zero withdrawal.
- ❖ Invalid amount.
- ❖ ATM has insufficient cash.
- ❖ Network failure.
- ❖ Card removal during transaction.

IMPORTANCE OF PROCESS CONSISTENCY

- ❖ A defined process ensures that all testers follow the same method. It prevents important scenarios from being forgotten and makes test results easier to compare.

CONCLUSION

- ❖ A structured SDLC-based testing process helps identify defects early and prevents inconsistent ATM behavior. It improves reliability, transaction accuracy, security, and customer confidence.

3. ERRORS, DEFECTS AND FAILURES IN LOGIN AND TRANSACTIONS

INTRODUCTION

- ❖ The terms **error**, **defect**, and **failure** are related but have different meanings. Understanding these terms is important for proper software testing and defect reporting.

ERROR

- ❖ An **error** is a human mistake made during activities such as requirements analysis, design, coding, or testing.
- ❖ For example, a developer misunderstands the login requirement and writes code that accepts an incorrect password under certain conditions. The developer's misunderstanding is an error.

DEFECT

- ❖ A **defect** is a flaw in the software caused by an error.
- ❖ For example, the login program contains a condition that incorrectly accepts a wrong password. This incorrect code is a defect.

FAILURE

- ❖ A **failure** occurs when the software actually behaves incorrectly during execution.
- ❖ For example, a user enters:
 - ❖ Username: customer01
 - ❖ Password: Wrong123
- ❖ The system allows the user to log in even though the password is incorrect. This visible incorrect behavior is a failure.

RELATIONSHIP

- ❖ The relationship can be represented as:
 - ❖ **Human Error** → **Software Defect** → **Execution** → **Failure**
- ❖ For example:
 - ❖ **Developer makes coding mistake** → **Incorrect login condition exists** → **User enters wrong password** → **System allows login**.

TRANSACTION EXAMPLE

- ❖ Suppose a customer transfers ₹5,000.
- ❖ The requirement says the account should be debited exactly ₹5,000.
- ❖ If the programmer accidentally writes logic that debits ₹50,000, the coding mistake creates a **defect**. When the transaction executes and ₹50,000 is deducted, the customer experiences a **failure**.

EFFECT OF IMPROPER UNDERSTANDING

- ❖ If testers do not understand these terms, defect reports may be unclear. A report saying "login error" does not explain whether the problem is a requirement mistake, coding defect, or observed failure.

IMPROVING DEFECT REPORTs

- ❖ A good defect report should contain:
- ❖ Defect ID
- ❖ Module
- ❖ Environment
- ❖ Preconditions
- ❖ Steps to reproduce
- ❖ Test data
- ❖ Expected result
- ❖ Actual result
- ❖ Severity
- ❖ Priority
- ❖ Screenshots/logs
- ❖ Build version

CONCLUSION

- ❖ Understanding error, defect, and failure improves communication between testers, developers, and managers. Clear terminology results in faster defect analysis and better software quality.

4. DEFECT CLUSTERING AND TEST CASE PRIORITIZATION

Introduction

- ❖ **Defect clustering** is an important testing principle stating that a small number of modules often contain a large number of defects. Therefore, testers should not always distribute testing effort equally across all modules.

Defect Clustering

- ❖ For example, an e-commerce application may contain:
 - ❖ Login module
 - ❖ Product search
 - ❖ Shopping cart
 - ❖ Payment
 - ❖ Reports
- ❖ Suppose 70% of previous defects were found in the payment and shopping-cart modules. These areas should receive greater testing attention.

Other Testing Principles

- ❖ Important testing principles include:

Testing Shows Presence of Defects

- ❖ Testing can demonstrate that defects exist, but it cannot prove that software has absolutely no defects.

Exhaustive Testing Is Impossible

- ❖ It is usually impossible to test every input and every combination. Therefore, testers prioritize important scenarios.

Early Testing

- ❖ Testing should start as early as possible in the SDLC.

Pesticide Paradox

- ❖ Repeatedly executing the same tests may stop finding new defects. Test cases should therefore be reviewed and updated.

Test Case Prioritization

- ❖ Test cases can be prioritized using:
 - ❖ Business importance
 - ❖ Risk
 - ❖ Previous defect history

- ❖ Customer impact
- ❖ Security importance
- ❖ Frequency of use

For example:

- ❖ | Test Case | Priority | |---|---| | Successful payment | High | | Payment failure | High | | Refund | High | | Changing profile picture | Low |

Practical Example

- ❖ If a banking application has repeated defects in fund transfer, the team should test fund transfer before low-risk features such as changing the profile theme.

Benefits

- ❖ Defect clustering helps the team:
- ❖ Focus limited resources.
- ❖ Detect critical defects earlier.
- ❖ Reduce testing time.
- ❖ Improve risk coverage.
- ❖ Increase confidence in important modules.

Conclusion

- ❖ Defect clustering and other testing principles help testers make intelligent decisions about where to spend testing effort. Risk-based prioritization provides better quality than treating every test case equally.

5. DEFECT CLASSIFICATION AND DEFECT REPOSITORY

Introduction

- ❖ A **defect repository** is a central location where software defects are recorded, tracked, analyzed, and maintained. Proper use of a repository helps organizations avoid repeating the same mistakes.

How Defects Originate

- ❖ Defects can originate during:
 - ❖ Requirement analysis
 - ❖ System design
 - ❖ Coding
 - ❖ Database development
 - ❖ Integration
 - ❖ Configuration
 - ❖ Testing
- ❖ For example, an ambiguous requirement can later result in incorrect functionality.

Defect Classification

- ❖ Defects can be classified as:
 - ❖ Functional defects
 - ❖ Performance defects
 - ❖ Security defects
 - ❖ Usability defects
 - ❖ Interface defects
 - ❖ Database defects
 - ❖ Compatibility defects
 - ❖ Requirement defects
 - ❖ Design defects
 - ❖ Coding defects
- ❖ They can also be classified according to **severity and priority**.

Using Repository Data

- ❖ Suppose the repository shows that every release has repeated defects related to incorrect date calculations.
- ❖ Testers can create additional tests for:
 - ❖ Leap years
 - ❖ Different months
 - ❖ Month-end dates

- ❖ Year-end processing
- ❖ Invalid dates
- ❖ Historical defect information therefore becomes an input for future test design.

Preventing Recurring Defects

- ❖ The company should:
- ❖ Perform root-cause analysis.
- ❖ Identify frequently recurring defect categories.
- ❖ Update regression suites.
- ❖ Improve coding standards.
- ❖ Conduct requirement and code reviews.
- ❖ Add automation for repeated failures.
- ❖ Share lessons learned with developers.

Example

- ❖ If three releases contain the same payment-calculation defect, the repository should highlight this pattern. The team can introduce automated payment regression tests and review the related business logic.

Conclusion

- ❖ A defect repository should not be treated as a simple list of bugs. It should be used as a **knowledge base** for test planning, root-cause analysis, prevention, and continuous improvement. This reduces recurring defects and improves software quality.

6. MOBILE BANKING APPLICATION CRASHES DURING FUND TRANSFER

Introduction

- ❖ A crash during fund transfer is a serious defect because it may affect money, account balances, and customer trust. Testing should be systematic to identify the root cause instead of repeatedly testing the same successful scenario.

Possible Causes

- ❖ The crash may originate from:
- ❖ Incorrect application logic
- ❖ API failure
- ❖ Network interruption
- ❖ Invalid input handling
- ❖ Database failure
- ❖ Memory problems
- ❖ Authentication timeout
- ❖ Concurrent transactions
- ❖ Unexpected server response

Systematic Testing Approach

Step 1: Reproduce the Failure

- ❖ The tester records:
- ❖ Device model
- ❖ Operating system
- ❖ App version
- ❖ Network condition
- ❖ Account details used for testing
- ❖ Transaction amount
- ❖ Exact steps

Step 2: Analyze Logs

- ❖ Crash logs and server logs are examined to determine where the failure occurred.

Step 3: Create Test Scenarios

- ❖ Test cases should cover:
- ❖ Normal transfer

- ❖ Large transfer
- ❖ Insufficient balance
- ❖ Invalid beneficiary
- ❖ Network interruption
- ❖ Session timeout
- ❖ Multiple transfers
- ❖ Slow network
- ❖ Server unavailable

Step 4: Negative Testing

- ❖ The tester intentionally supplies invalid conditions to determine whether the application handles them safely.

Step 5: Regression Testing

- ❖ After fixing the crash, all related fund-transfer functions are tested again.

Example

- ❖ Suppose the application crashes when the network disconnects immediately after pressing **Transfer**. Testing should verify whether:
 - ❖ Money was deducted.
 - ❖ Transaction was completed.
 - ❖ Transaction status was displayed.
 - ❖ Duplicate transfer can occur after retry.

Improvements

- ❖ The organization should introduce:
 - ❖ Automated API tests.
 - ❖ Crash monitoring.
 - ❖ Network-condition testing.
 - ❖ Regression suites.
 - ❖ Code reviews.
 - ❖ Risk-based testing.
 - ❖ Performance and security testing.

Conclusion

- ❖ Systematic testing helps identify not only the visible crash but also its root cause and related transaction risks. This leads to safer and more reliable mobile banking applications.

7. TESTING AND DEBUGGING

Introduction

- ❖ Testing and debugging are related but different activities. **Testing identifies failures or defects, while debugging investigates the cause and fixes the defect.**

Testing

- ❖ Testing is performed to determine whether software meets specified requirements and to find failures.
- ❖ For example, a tester enters an incorrect PIN into an ATM. If the ATM allows access, testing has identified a failure.

Debugging

- ❖ Debugging is normally performed by developers to locate the cause of the failure and correct the software.
- ❖ The developer may discover that the PIN comparison condition is incorrectly implemented.

Difference

- ❖ | Testing | Debugging | |---|---| | Finds failures | Finds causes | | Usually performed by testers | Usually performed by developers | | Compares expected and actual results | Examines code, logs and execution | | Reports defects | Fixes defects |

Problems When They Are Confused

- ❖ If testers try to directly fix defects instead of reporting them, important information may be lost. Similarly, developers may assume that successful execution means there are no defects.

Structured Approach

- ❖ A good process is:

Test → Detect Failure → Report Defect → Analyze → Debug → Fix → Retest → Regression Test

Example

- ❖ A banking application displays ₹8,000 instead of ₹10,000.
- ❖ The tester reports the incorrect balance with evidence. The developer investigates the database update logic and identifies the cause. After fixing it, the tester retests the balance and performs regression testing.

Conclusion

- ❖ Testing and debugging have different responsibilities but work together. Proper coordination between

both processes improves defect resolution speed and software quality



8. CROSS-BROWSER TESTING FOR AN E-COMMERCE PLATFORM

Introduction

- ❖ A web application may behave differently across browsers because browsers can interpret HTML, CSS, JavaScript, and other technologies differently. Therefore, systematic cross-browser testing is required.

Why Defects Occur

- ❖ Defects can originate because of:
 - ❖ Browser rendering differences.
 - ❖ Different JavaScript support.
 - ❖ CSS compatibility problems.
 - ❖ Different screen resolutions.
 - ❖ Browser-specific APIs.
 - ❖ Different browser versions.
 - ❖ Mobile and desktop differences.

Testing Strategy

- ❖ The team should identify the browsers used by customers, such as:
 - ❖ Chrome
 - ❖ Firefox
 - ❖ Edge
 - ❖ Safari
- ❖ Testing should cover important combinations of browser, operating system, screen size, and device.

Important Test Areas

- ❖ Test:
 - ❖ Login
 - ❖ Product search
 - ❖ Product display
 - ❖ Shopping cart
 - ❖ Checkout
 - ❖ Payment
 - ❖ Forms
 - ❖ Images
 - ❖ Menus
 - ❖ Responsive design

Tools and Techniques

- ❖ Tools such as browser automation frameworks can execute the same tests across multiple browsers. Cloud-based browser testing platforms can also provide access to different browser and operating-system combinations.

Example

- ❖ Suppose the **Add to Cart** button works in Chrome but is not visible in Safari.
- ❖ The tester should record:
 - ❖ Browser and version.
 - ❖ Operating system.
 - ❖ Page URL/build.
 - ❖ Steps to reproduce.
 - ❖ Screenshot.
 - ❖ Expected behavior.
 - ❖ Actual behavior.
- ❖ Developers can then investigate the CSS or JavaScript causing the problem.

Conclusion

- ❖ Cross-browser testing improves compatibility, usability, and customer experience. Combining manual testing, automation, responsive testing, and browser coverage provides better reliability.

9. DEFECT CLASSIFICATION – PAYROLL SALARY CALCULATION

Introduction

- ❖ A payroll system must calculate salary accurately. A logical error in salary calculation can cause employees to receive incorrect payments. Defect classification helps identify the nature and source of such problems.

Types of Defects

- ❖ The salary problem may be classified as a **functional defect** because the system does not perform its required function correctly.
- ❖ Depending on its origin, it may also be a:
 - ❖ Requirement defect
 - ❖ Design defect
 - ❖ Coding defect
 - ❖ Calculation/logic defect
 - ❖ Database defect
 - ❖ Integration defect

Example

- ❖ Suppose:
- ❖ **Basic Salary = ₹30,000**
- ❖ **Allowance = ₹5,000**
- ❖ Expected gross salary:
- ❖ **₹30,000 + ₹5,000 = ₹35,000**
- ❖ But the system calculates ₹30,000 because the allowance is not included.
- ❖ This is a functional and logical defect.

Testing Techniques

- ❖ Testers can use:

Equivalence Partitioning

- ❖ Test different salary ranges.

Boundary Value Analysis

- ❖ Test values around salary limits and tax slabs.

Decision Tables

- ❖ Test combinations of salary, allowance, deductions, tax, and employee category.

Data-Driven Testing

- ❖ Execute many employee salary combinations automatically.

Prevention

- ❖ Defects can be prevented through:
 - ❖ Requirement reviews.
 - ❖ Design reviews.
 - ❖ Code reviews.
 - ❖ Unit testing.
 - ❖ Automated calculation tests.
 - ❖ Regression testing.
 - ❖ Independent test data verification.

Conclusion

- ❖ Defect classification helps the team understand where and why salary errors occur. Combining classification with systematic test design reduces calculation errors and improves payroll reliability.

10. STRUCTURED TEST CASE DESIGN INSTEAD OF RANDOM TESTING

Introduction

- ❖ Random testing without planning may miss important scenarios. Testing as an engineering activity requires **systematic test case design** based on requirements, risk, and appropriate testing techniques.

Importance of Test Cases

- ❖ A good test case defines:
 - ❖ Test case ID
 - ❖ Objective
 - ❖ Preconditions
 - ❖ Input
 - ❖ Test steps
 - ❖ Expected result
 - ❖ Actual result
 - ❖ Status
 - ❖ Requirement reference
- ❖ This makes testing repeatable and measurable.

Suitable Techniques

Equivalence Partitioning

- ❖ Inputs are divided into valid and invalid groups.

Boundary Value Analysis

- ❖ Values at and around boundaries are tested.

Decision Table Testing

- ❖ Useful when output depends on multiple conditions.

State Transition Testing

- ❖ Useful for systems whose behavior changes according to state.

Use-Case Testing

- ❖ Tests complete business scenarios.

Risk-Based Testing

- ❖ High-risk functionality receives more testing effort.

Example

- ❖ For a login system, random testing may test only:

Correct username + correct password.

- ❖ Structured testing also checks:

Correct username + wrong password.

Wrong username + correct password.

Both incorrect.

Empty username.

Empty password.

Locked account.

Expired password.

Multiple failed attempts.

Session timeout.

Benefits

- ❖ Structured test cases:
- ❖ Increase coverage.
- ❖ Reduce missed scenarios.
- ❖ Make testing repeatable.
- ❖ Support regression testing.
- ❖ Improve defect reporting.
- ❖ Make test results measurable.

Conclusion

- ❖ Random testing can be useful as exploratory testing, but it should not replace structured test design. Engineering-based test case design provides better coverage and improves defect detection.

11. DATA INCONSISTENCY IN A HEALTHCARE APPLICATION

Introduction

- ❖ Healthcare applications handle important patient information such as medical history, prescriptions, appointments, and billing. Data inconsistency between modules can therefore create serious problems.

Possible Sources of Defects

- ❖ Defects may originate from:
 - ❖ Incorrect requirements.
 - ❖ Database design problems.
 - ❖ Incorrect data mapping.
 - ❖ API integration errors.
 - ❖ Incorrect update logic.
 - ❖ Concurrency problems.
 - ❖ Poor validation.
 - ❖ Incorrect data transformation.

Example

- ❖ Suppose a patient's name is updated in the registration module but the old name appears in the billing module. This indicates a data synchronization or integration defect.

Systematic Testing

- ❖ Testers should verify data across:
 - ❖ Patient registration.
 - ❖ Appointment management.
 - ❖ Medical records.
 - ❖ Pharmacy.
 - ❖ Billing.
 - ❖ Reports.
- ❖ Important checks include:
 - ❖ Same patient ID across modules.
 - ❖ Correct data format.
 - ❖ Correct database values.
 - ❖ Correct API responses.
 - ❖ Updates reflected in dependent modules.

Defect Tracking

- ❖ Each defect should contain:
 - ❖ Patient/module identifier using safe test data.
 - ❖ Environment.
 - ❖ Steps.
 - ❖ Expected result.
 - ❖ Actual result.
 - ❖ Screenshots/logs.
 - ❖ Severity.
 - ❖ Related requirement.
- ❖ Sensitive patient information should not be unnecessarily included in defect reports.

Conclusion

- ❖ Systematic integration and data-integrity testing ensures that information remains accurate across healthcare modules. Proper defect tracking also helps developers identify recurring integration problems.

12. REPEATED DEFECTS AND DEFECT REPOSITORY

Introduction

- ❖ Repeated defects indicate that an organization is testing and fixing problems but not learning from previous failures. A defect repository can help identify these patterns and prevent recurrence.

Repository Information

- ❖ A useful repository contains:
- ❖ Defect ID
- ❖ Module
- ❖ Description
- ❖ Severity
- ❖ Priority
- ❖ Root cause
- ❖ Status
- ❖ Release
- ❖ Environment
- ❖ Resolution
- ❖ Related test cases

Identifying Patterns

- ❖ Suppose repository analysis shows:
- ❖ 30% defects in payment.
- ❖ 20% defects in login.
- ❖ Many defects occur during integration.
- ❖ Several defects are caused by incorrect input validation.
- ❖ This information can guide future test planning.

Using Repository Data

- ❖ Historical defects should be converted into:
- ❖ New regression tests.
- ❖ Additional negative tests.
- ❖ Automation scripts.
- ❖ Code-review checklists.
- ❖ Requirement-review guidelines.

Example

- ❖ If previous releases repeatedly had duplicate-payment defects, the next release should include tests for:

- ❖ Double-clicking payment.
- ❖ Network retry.
- ❖ Browser refresh.
- ❖ Timeout and retry.
- ❖ Duplicate API requests.

Improvements

- ❖ The organization should perform:
- ❖ Root-cause analysis.
- ❖ Defect trend analysis.
- ❖ Periodic repository reviews.
- ❖ Regression-suite updates.
- ❖ Knowledge sharing.
- ❖ Preventive action tracking.

Conclusion

- ❖ A defect repository becomes valuable when its information is analyzed and used for future testing. It changes testing from reactive bug fixing into **preventive quality improvement**.

13. WEB LEARNING SYSTEM FAILS DURING PEAK USAGE

Introduction

- ❖ A learning system may work correctly under normal usage but fail when thousands of students access it simultaneously. This indicates gaps in performance and capacity testing.

Testing Gaps

- ❖ The team may have tested:
 - ❖ Login with a few users.
 - ❖ Course access.
 - ❖ Assignment submission.
- ❖ But they may not have tested realistic peak conditions.

Test Planning

- ❖ Performance requirements should be identified early. The test plan should specify:
 - ❖ Expected number of users.
 - ❖ Peak traffic.
 - ❖ Response-time requirements.
 - ❖ Concurrent sessions.
 - ❖ Database load.
 - ❖ Network conditions.
 - ❖ Acceptable failure rate.

Performance Testing

- ❖ Important techniques include:

Load Testing

- ❖ Checks system behavior under expected user load.

Stress Testing

Checks behavior beyond normal capacity.

Spike Testing

- ❖ Checks sudden increases in users.

Endurance Testing

- ❖ Checks system behavior over a long period.

Example

- ❖ Suppose a learning platform normally has 1,000 users but 10,000 students access it just before an examination.
- ❖ The system should be tested with realistic concurrent users before deployment.

Coverage

- ❖ Testing should include:
 - ❖ Login.
 - ❖ Course access.
 - ❖ Video streaming.
 - ❖ Assignment upload.
 - ❖ Quiz submission.
 - ❖ Result viewing.

Conclusion

- ❖ Performance testing should be included in test planning rather than performed only after failures occur. Proper load and stress testing improves reliability during peak usage and prevents major production failures.

❖ 14. EARLY TESTER INVOLVEMENT IN SDLC

Introduction

- ❖ Testers should not be involved only after development is completed. Early tester involvement helps identify problems before they become expensive to fix.

Role During Requirements

- ❖ Testers can review requirements for:
 - ❖ Missing conditions.
 - ❖ Ambiguity.
 - ❖ Contradictions.
 - ❖ Incorrect assumptions.
 - ❖ Unclear acceptance criteria.

Role During Design

- ❖ Testers can review design documents and identify:
 - ❖ Integration risks.
 - ❖ Data-flow problems.
 - ❖ Security concerns.
 - ❖ Difficult-to-test areas.

Role During Development

- ❖ Testers can support:
 - ❖ Unit-test planning.
 - ❖ Test-data preparation.
 - ❖ Code reviews.
 - ❖ API testing.
 - ❖ Automation planning.

Practical Example

- ❖ Suppose a banking requirement says:
 - ❖ **"Users can transfer money up to the allowed limit."**
 - ❖ This requirement is ambiguous because the exact limit is not specified.
 - ❖ An early tester can ask:
 - ❖ What is the limit?
 - ❖ Is it per transaction or per day?

- ❖ What happens when the limit is exceeded?
- ❖ What message should be displayed?
- ❖ These questions prevent future defects.

Benefits

- ❖ Early involvement provides:
- ❖ Early defect detection.
- ❖ Lower correction cost.
- ❖ Better requirements.
- ❖ Improved testability.
- ❖ Better collaboration.
- ❖ Fewer misunderstandings.

Team Collaboration

- ❖ Developers, testers, business analysts, and product owners should communicate continuously through reviews, planning meetings, and shared documentation.

Conclusion

- ❖ Early tester involvement supports the principle of "**shift-left testing.**" It improves quality by preventing defects rather than finding all defects after development is complete.

15. INCOMPLETE TEST COVERAGE

Introduction

- ❖ A system may produce correct results for some scenarios but fail in others when test coverage is incomplete. Good testing requires coverage of normal, abnormal, boundary, and risk-related conditions.

Causes of Incomplete Coverage

- ❖ Missing requirements.
- ❖ Poor test-case design.
- ❖ Testing only positive scenarios.
- ❖ Lack of boundary testing.
- ❖ Ignoring error conditions.
- ❖ Limited test data.
- ❖ Time pressure.

Example

- ❖ Suppose an ATM withdrawal works correctly for ₹1,000.
- ❖ Testing only this scenario does not prove the withdrawal function is reliable.
- ❖ Additional cases should include:
 - ❖ ₹0
 - ❖ Negative amount
 - ❖ Minimum allowed amount
 - ❖ Maximum allowed amount
 - ❖ Amount above daily limit
 - ❖ Insufficient balance
 - ❖ Invalid denomination
 - ❖ ATM out of cash
 - ❖ Network failure

Improving Test Design

- ❖ Testers should use:
 - ❖ Equivalence partitioning.
 - ❖ Boundary value analysis.
 - ❖ Decision tables.
 - ❖ State transition testing.
 - ❖ Use-case testing.
 - ❖ Risk-based testing.
 - ❖ Requirement traceability.

Coverage

- ❖ Coverage can be evaluated using:
- ❖ Requirement coverage.
- ❖ Test-case coverage.
- ❖ Code coverage.
- ❖ Risk coverage.
- ❖ Browser/device coverage.

Validation

- ❖ Expected results should be clearly defined before execution. Actual results should then be compared against them.

Conclusion

- ❖ Complete coverage does not mean testing every possible combination. It means systematically selecting important scenarios so that major risks and requirements are adequately tested.

16. IMPROVING DEFECT REPORTING IN A LOGISTICS APPLICATION

Introduction

- ❖ Poor defect reporting can delay defect identification and resolution. In a logistics application, unclear defects can affect delivery tracking, shipment status, warehouse operations, and customer communication.

Importance of Clear Defect Definitions

- ❖ A defect should describe a deviation between the **expected result and actual result**.
- ❖ For example:
- ❖ **Poor report:** "Tracking is not working."
- ❖ **Better report:** "When a valid tracking number is entered, the system displays 'Invalid Tracking Number' instead of showing the shipment status."
- ❖ The second report is much easier to understand.

Structured Defect Report

- ❖ A good report should include:
- ❖ Defect ID
- ❖ Title
- ❖ Module
- ❖ Build/version
- ❖ Environment
- ❖ Preconditions
- ❖ Steps to reproduce
- ❖ Test data
- ❖ Expected result
- ❖ Actual result
- ❖ Severity
- ❖ Priority
- ❖ Attachments/logs

Example

- ❖ **Title:** Shipment status not updated after delivery confirmation.
- ❖ **Steps:**
- ❖ Create a shipment.
- ❖ Mark it as delivered.

- ❖ Open customer tracking page.
- ❖ **Expected:** Status should display "Delivered."
- ❖ **Actual:** Status remains "Out for Delivery."

Benefits

- ❖ Structured reports:
- ❖ Reduce communication gaps.
- ❖ Speed up debugging.
- ❖ Prevent duplicate reports.
- ❖ Help managers track trends.
- ❖ Improve traceability.

Conclusion

- ❖ Clear and standardized defect reporting improves communication between testers and developers. It helps defects get reproduced, analyzed, fixed, and verified more efficiently.

17. MAINTAINING CONSISTENCY IN TESTING ACROSS MODULES

Introduction

- ❖ When different testers use different methods, testing becomes inconsistent. Some modules may receive detailed testing while others receive very little. A process-based testing approach solves this problem.

Standardized Testing Process

- ❖ The organization should establish a common process:
- ❖ **Requirement Analysis → Test Planning → Test Design → Test Execution → Defect Reporting → Retesting → Regression Testing → Test Closure**

Common Standards

- ❖ All modules should use standardized:
- ❖ Test-case templates.
- ❖ Defect-report formats.
- ❖ Severity definitions.
- ❖ Priority definitions.
- ❖ Naming conventions.
- ❖ Entry and exit criteria.
- ❖ Review processes.

Example

- ❖ Suppose the banking application has three modules:
- ❖ Login
- ❖ Fund Transfer
- ❖ Account Statement
- ❖ All teams should use the same test-case format and defect classification system.

Test Reviews

- ❖ Test cases should be reviewed before execution to ensure that important requirements and risks are covered.

Automation

- ❖ Common regression tests can be automated so that every module is tested consistently after a new build.

Metrics

- ❖ Management can monitor:
- ❖ Number of test cases.
- ❖ Pass/fail percentage.
- ❖ Defect density.
- ❖ Defect severity.
- ❖ Test coverage.
- ❖ Reopened defects.

Conclusion

- ❖ A standardized testing process makes testing predictable and repeatable. It reduces variation between teams and improves overall software quality.

18. EDGE CASES AND BOUNDARY VALUE ANALYSIS

Introduction

- ❖ Edge cases are unusual or extreme conditions that may not occur frequently but can cause serious failures. Many production defects occur because these conditions were not tested.

Why Edge Cases Are Missed

- ❖ They may be missed because testers focus mainly on normal scenarios.
- ❖ For example, a system may be tested with ages such as 25 and 40 but not with:
 - ❖ Minimum allowed age.
 - ❖ Maximum allowed age.
 - ❖ One value below the minimum.
 - ❖ One value above the maximum.

Boundary Value Analysis

- ❖ Boundary Value Analysis focuses on values at and around limits.
- ❖ Suppose an online banking system allows transfers from ₹100 to ₹50,000.
- ❖ Important tests include:
 - ❖ ₹99
 - ❖ ₹100
 - ❖ ₹101
 - ❖ ₹49,999
 - ❖ ₹50,000
 - ❖ ₹50,001
- ❖ These tests are more useful than randomly selecting many values.

Other Edge Cases

- ❖ Testers should consider:
 - ❖ Empty input.
 - ❖ Maximum-length input.
 - ❖ Special characters.
 - ❖ Zero.
 - ❖ Negative values.
 - ❖ Very large values.
 - ❖ Expired sessions.



- ❖ Network interruption.
- ❖ Duplicate requests.

Practical Example

- ❖ If a system accepts a username of 6–20 characters, test:
- ❖ **5, 6, 7, 19, 20, and 21 characters.**

Conclusion

- ❖ Boundary and edge-case testing helps identify defects that normal testing may miss. It is especially important for banking, payroll, healthcare, and other systems where incorrect boundary handling can have serious consequences.

19. DEFECT DOCUMENTATION AND DEFECT REPOSITORY

Introduction

- ❖ A company without proper defect documentation may lose information about previous problems. This makes it difficult to track defects, identify recurring problems, and measure software quality.

Defect Repository

- ❖ A defect repository is a centralized system containing information about reported defects throughout the software lifecycle.

Important Information

- ❖ Each defect should contain:
- ❖ Unique defect ID.
- ❖ Title.
- ❖ Description.
- ❖ Module.
- ❖ Severity.
- ❖ Priority.
- ❖ Environment.
- ❖ Steps to reproduce.
- ❖ Expected result.
- ❖ Actual result.
- ❖ Assigned developer.
- ❖ Status.
- ❖ Root cause.
- ❖ Resolution.
- ❖ Release information.

Benefits

- ❖ **Tracking**
- ❖ The team can know whether a defect is:
- ❖ **New → Assigned → In Progress → Fixed → Retest → Closed**

Historical Analysis

- ❖ Previous defects can be analyzed to identify:
- ❖ High-defect modules.

- ❖ Common root causes.
- ❖ Repeated failures.
- ❖ Quality trends.

Test Planning

- ❖ Historical defects can be converted into regression test cases.

Example

- ❖ Suppose the repository shows that the same payment failure occurred in three releases.
- ❖ The team can:
 - ❖ Find the root cause.
 - ❖ Add a regression test.
 - ❖ Automate the test.
 - ❖ Review related code.
 - ❖ Monitor the defect in future releases.

Documentation Improvements

- ❖ The company should use standardized defect templates, mandatory fields, clear severity definitions, and regular repository reviews.

Conclusion

- ❖ Good defect documentation provides traceability and organizational learning. A well-maintained repository helps the team move from repeatedly fixing defects to preventing them.

20. REDESIGNING TESTING FOR REPEATED BANKING TRANSACTION FAILURES

Introduction

- ❖ Banking transaction failures can have serious financial and legal consequences. If failures occur repeatedly, the testing process is probably not systematic enough. Testing should therefore be redesigned as a structured engineering activity.

Step 1: Requirement Analysis

- ❖ The team should identify all transaction requirements, including:
 - ❖ Valid transactions.
 - ❖ Transaction limits.
 - ❖ Account balance rules.
 - ❖ Authentication.
 - ❖ Transaction status.
 - ❖ Failure handling.
 - ❖ Refund/reversal behavior.

Step 2: Risk Analysis

- ❖ High-risk areas such as fund transfer and payment processing should receive higher testing priority.

Step 3: Test Design

- ❖ Suitable techniques include:

Equivalence Partitioning

- ❖ Separate valid and invalid transaction inputs.

Boundary Value Analysis

- ❖ Test transaction limits.

Decision Tables

- ❖ Test combinations such as sufficient/insufficient balance and valid/invalid accounts.

State Transition Testing

- ❖ Check transaction states such as:
 - ❖ **Initiated → Processing → Successful/Failed/Reversed**

Negative Testing

- ❖ Test failures such as network loss, invalid accounts, expired sessions, and duplicate requests.

Step 4: Automation

- ❖ Automate stable regression tests for:
 - ❖ Login.
 - ❖ Balance verification.
 - ❖ Fund transfer.
 - ❖ Payment.
 - ❖ Transaction history.
 - ❖ Reversal.
- ❖ API automation can test transaction services quickly, while UI automation can verify important end-to-end flows.

Step 5: Defect Management

- ❖ Each defect should contain clear reproduction steps, expected and actual results, severity, logs, and transaction information using safe test data.

Practical Example

- ❖ Suppose a customer clicks **Transfer** and the network disconnects.
- ❖ The system should not create two transactions when the customer retries.
- ❖ A structured test can verify:
 - ❖ Start transfer.
 - ❖ Interrupt network.
 - ❖ Restore network.
 - ❖ Retry transfer.
 - ❖ Check account balance.
 - ❖ Check transaction history.
- ❖ This can detect duplicate transaction defects.

Step 6: Regression Testing

- ❖ After fixing a transaction defect, related transaction scenarios must be retested. Automated regression testing should be executed for every important release.

Conclusion

- ❖ A structured banking testing approach improves defect detection by combining **risk-based testing, systematic test design, automation, negative testing, boundary testing, defect analysis, and regression testing**. This reduces repeated transaction failures and improves reliability, security, customer trust, and overall software quality.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand